- **QueryArrayCached:** Returns the cached result for a previous query run. If the result is not already present in the cache, this function will save the result set in the cache as well.
- **InstantiateDbRow:** Instantiates a single DB row as an object. It also expands the object's properties to cover any related object, based on what was requested for expansion (discussed later).
- **InstantiateDbResult**: Compiles a list of objects instantiated by *InstantiateDbRow* to create the result array.
- **CountBy*XYZ*:** Returns the number of results for a given value of a column (represented by 'XYZ'). It is noteworthy that *CountByXYZ* methods are created for only those columns on which an index has been defined. The 'XYZ' part is replaced by the name of the column.
- **LoadBy*XYZ*:** These methods are created for any column on which a unique index is defined. As you can guess, again, 'XYZ' is replaced by the name of the column.
- **LoadArrayBy*XYZ*:** These methods are created for any column with a 'non-unique' index on them.
- **Save:** Saves the object. Saving the object will either cause an update (if you are saving a modified object) or create a new object (if you created a new object). You may also force an insertion even if the object is to be updated (you have to make sure that the condition when you do this will not fail on the database level). This also deletes the old cached object (if it was already there).
- **BuildQueryStatement:** This function actually builds the final query. You do not use it directly.

All these features are built into the ORM-Gen class files (located in the *includes/model/generated* directory in the extracted directory—the code-base root directory). One does not have to use them directly. Instead, the ORM Class files, which are located in *includes/model* have to be used, which are sub-classes of their respective 'Gen' classes. During the Codegen process, the ORM-Gen class files are overwritten, but the ORM class files are not (they, however, are created if they do not exist already). This allows you to write class-specific methods in the ORM classes, which makes sure that your custom methods are not overwritten on subsequent Codegens!

## Nodes and expansion

The functions just mentioned make it quite easy to work with the objects of a table. But what about nesting the queries? Well, that feature is available too. The fact is, all tables and columns are treated as nodes. Nodes are connected with each other depending on the foreign keys in place. When running a query, you can cause the resulting objects in the set to *expand* the column(s) into another object of the table with which the column establishes a foreign key relationship. Have a look at *http://bit.ly/W4Ryer* to understand how this is done. Do not forget to view the source code.

## Creating forms for data entry

Codegen deals with databases, and databases store data. It is obvious that your Web application is going to need some data before it really becomes useful. No matter what type of Web app you are going to build, entering data is a mandate, at least for testing. PhpMyAdmin or similar tools allow you to enter data into your database, but then it is not always safe to use them; one mistaken click of a button and you can end up deleting a complete table! Well, you can use them while starting off with the application, but as the complexity of the schema grows, they too become cumbersome to use.

Speaking of 'cumbersome', would it not be good if someone just created a form automatically, with you needing to only feed data to the DB? How about if someone created reusable, ready-to-use controls for your database columns, so you could use them in your Web app without writing the code for it? Does it sound like you need to pay money to people employed for the job? To me, it does not.

QCubed's Codegen goes a step forward, and creates forms that allow you to insert data. The good thing is that these forms are made of modular controls. A form is built for each table. Each form in itself is a *panel* which contains *MetaControls* (explained later) and they, in turn, contain input controls for each column of the table. The pages that contain the data entry forms are called 'draft' pages—so named because they can be used as a draft to help you build your applications. The forms come with the following benefits:

1. **Form validation**—This prevents you from entering invalid data. For example, if you try to enter characters in a text-box that is supposed to receive only integers, it will show an error. This function implements the *'maxlength'* attribute of text input controls to prevent you from entering extra-long text for *varchar* columns. A NOT NULL field cannot be left empty, and so on. If done manually, creating the validation system will take a lot of time.
2. **Foreign key controls—**If you have a foreign key column in a table, then the respective input control for that column in the 'Codegened' form would be a drop-down list (HTML *select* element) of values representing rows in the related table. Easier than referring to the table in another tab and entering data manually, right?
3. **CRUD—**You can use this to create a new row, update an old row or to delete one.

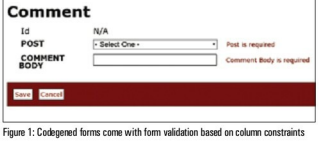


Figure 1: Codegened forms come with form validation based on column constraints